Módulo 1

Introducción a TypeScript

Curso: TypeScript

45 min

Objetivos de Aprendizaje

- Comprender qué es **TypeScript** y su propuesta de valor
- Conocer la **historia** y motivación de su creación
- Identificar los **beneficios** del tipado estático
- Reconocer **cuándo usar** TypeScript en proyectos

1. Contexto

Un caso que lo cambia todo

El Problema: La Aplicación que Nadie Quiere Mantener

```
src/
├── modules/
│   ├── auth/
│   │   ├── login.ts        // Función: validateCredentials()
│   │   ├── session.ts      // Función: createSession()
│   │   └── tokens.ts       // Función: generateToken()
│   ├── users/
│   │   ├── profile.ts      // Función: getUserProfile()
│   │   ├── settings.ts     // Función: updateSettings()
│   │   └── preferences.ts  // Función: getPreferences()
│   ├── orders/
│   │   ├── checkout.ts     // Función: processOrder()
│   │   ├── cart.ts         // Función: addToCart()
│   │   └── history.ts      // Función: getOrderHistory()
│   └── ... (97 módulos más)
```

100 módulos × ~10 funciones × ~5 parámetros = **5000+ puntos de fricción**

El Error: ¿Qué Propiedades Tiene "user"?

auth/tokens.js (hace 6 meses)

```
function generateToken(user) {
  return {
    userId: user.id,
    exp: Date.now() + 3600000
  };
}
```

orders/history.js (hace 1 mes)

```
function getOrderHistory(user) {
  const token = generateToken(user.name);
  // ¿user.name? ¿user.email? ¿user.id?
}
```

El error solo aparece en producción.

Preguntas que TypeScript Responde

- ¿Cómo sabemos qué propiedades tiene `user`?
- ¿Cómo evitamos que alguien use `user.name` en lugar de `user.id`?
- ¿Cómo refactorizamos sin romper todo?
- ¿Cómo documentamos el código para los nuevos desarrolladores?

Conclusión Clave

En aplicaciones **grandes y con múltiples desarrolladores**:

- JavaScript no escala: errores en tiempo de ejecución
- La documentación se desactualiza rápidamente
- El refactoring es arriesgado sin tipos

TypeScript resuelve estos problemas añadiendo tipos estáticos.

2. ¿Qué es TypeScript?

TypeScript en 30 segundos

- **Superset de JavaScript**: todo código JS es válido en TS
- Añade **tipos estáticos opcionales**
- **Compila a JavaScript** estándar (ES3+)
- Compatible con **todo el ecosistema JavaScript**

Historia de TypeScript

Año	Hito
2012	Microsoft crea TypeScript para resolver problemas de escala
2014	TypeScript 1.0 publicado
2015	Angular adopta TypeScript (punto de inflexión)
2017-actualidad	Adoptado por VS Code, Airbnb, Netflix, y miles de proyectos

TypeScript Compila a JavaScript

```
1 // TypeScript (input)
2 function greet(name: string): string {
3     return `Hola, ${name}!`;
4 }
```

```
1 // JavaScript (output)
2 function greet(name) {
3     return "Hola, " + name + "!";
4 }
```

Nota: TypeScript se "elimina" en tiempo de ejecución. Los tipos solo existen en desarrollo.

Conclusión Clave

TypeScript = JavaScript + tipos estáticos opcionales

- No rompe compatibilidad con JS existente
- Añade seguridad en tiempo de desarrollo
- El código compilado es JavaScript estándar

Referencias:

- Sitio oficial de TypeScript
- Why TypeScript?

3. Beneficios de TypeScript

5 Beneficios Clave

Beneficio	Impacto
Detección temprana de errores	Errores en compilación, no en producción
Autocompletado inteligente	El IDE conoce los tipos
Documentación automática	Los tipos documentan el código
Refactoring seguro	Renombrar y mover con confianza
Mantenibilidad	Código más fácil de entender y evolucionar

Ejemplo: Autocompletado con Tipos

```
1  interface User {
2      id: number;
3      name: string;
4      email: string;
5      isActive: boolean;
6  }
7
8  function getUserById(id: number): User | undefined {
9      // ...
10 }
11
12 // Al escribir user. el IDE muestra:
13 user.    →  id, name, email, isActive
14         ←  Sin types: sin sugerencias
```

VS Code + TypeScript = Superpoderes

Conclusión Clave

TypeScript **mejora la experiencia de desarrollo**:

- Menos bugs en producción
- IDE más inteligente
- Código autodocumentado

4. Cuándo Usar TypeScript

Usa TypeScript Cuando:

- Proyectos medianos/grandes (+10k líneas)
- Equipos de múltiples desarrolladores
- Código con evolución frecuente
- Proyectos de larga duración

✓ Ideal

- Aplicaciones enterprise
- APIs y backend
- Frameworks (Angular, Next.js)

NO Uses TypeScript Cuando:

✗ Evitar

- Scripts simples y rápidos
- Prototipos/PoCs
- Aprendizaje de JavaScript

- Proyectos tiny (1-2 archivos)
- Deadline muy estricto
- Equipo sin experiencia en TS

Casos de Éxito

- **Angular**: Framework completo de Google
- **VS Code**: Editor de Microsoft (1.5M+ líneas en TS)
- **Airbnb, Netflix, Slack**: Aplicaciones a escala

Conclusión Clave

TypeScript vale la pena cuando:

- El proyecto va a crecer
- Hay más de 1 desarrollador
- Se necesita mantenibilidad a largo plazo

No es overkill para scripts pequeños.

Casos del Curso

Caso 1: Sistema de Gestión de Tareas

Tipo: Práctico - Labs por módulo

Descripción: Aplicación para gestionar tareas personales

Enfoque: Progresivo - Cada módulo añade complejidad

Caso 2: Sistema de Gestión de Biblioteca

Tipo: Integrador - Ejercicios en grupo

Descripción: Gestión de préstamos de libros

Enfoque: Trabajo en equipo - Múltiples dominios

Resumen

Puntos Clave

- **TypeScript = JavaScript + tipos estáticos opcionales**
- Detecta errores **antes de ejecutar** el código
- Mejora la **experiencia de desarrollo** y mantenibilidad
- Se **compila a JavaScript** estándar
- Ideal para proyectos grandes con múltiples desarrolladores

Referencias

Recursos Adicionales

- Documentación oficial de TypeScript
- TypeScript Playground
- Why TypeScript?

Gracias!

Módulo 1 Completado

Siguiente: Módulo 2 - Entorno de desarrollo